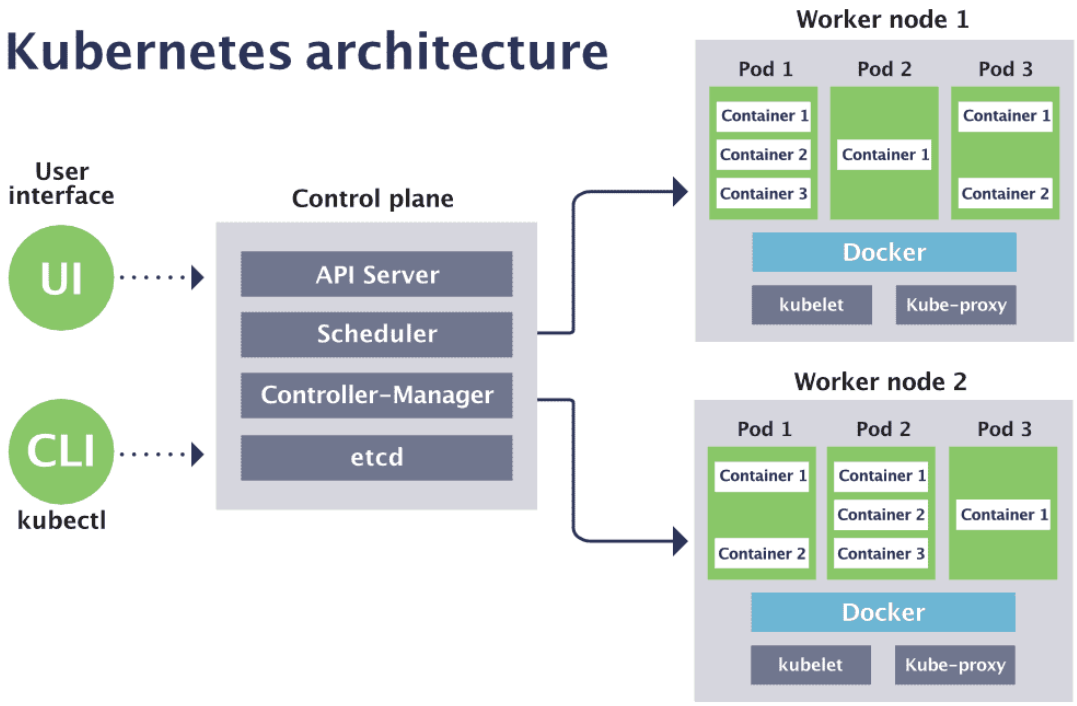
🔷 Kubernetes Architecture – From Control Plane to Pods 🔷

Kubernetes architecture is designed to manage containerized applications at scale with **high availability, self-healing, and zero downtime**.



STEP 1 User Interaction (How Everything Starts)

You never talk to worker nodes directly.

You interact with Kubernetes using:

- **CLI** → kubectl
- **UI** → Kubernetes Dashboard

Example:

kubectl apply -f app.yaml

✅ This request **always goes to the API Server first**
❌ It never talks directly to nodes or containers

STEP 2 Control Plane (The Brain of Kubernetes)

The **Control Plane** manages the entire cluster.

In **production**, it runs on **3 or more Linux machines** to avoid downtime.

Components inside the Control Plane

✅ **API Server (ENTRY POINT – MOST IMPORTANT)**

- Single gateway to the cluster
- Validates YAML
- Authenticates users
- Authorizes requests
- Communicates with ETCD and all components

📌 If API Server is down → **kubectl stops working**

✅ **ETCD (CLUSTER DATABASE)**

- Distributed **key–value store**
- Stores entire cluster state:
    - Pods
    - Nodes
    - Deployments
    - ConfigMaps
    - Secrets

✅ Must be backed up
✅ Always run in HA mode

✅ **Scheduler (DECISION MAKER)**

- Decides **where a Pod should run**
- Checks:
    - CPU / Memory
    - Node health
    - Taints & tolerations

- Affinity rules

📌 Scheduler only **decides** — it never runs pods

✅ **Controller Manager (STATE ENFORCER)**

- Continuously checks:
  Desired state vs Actual state
- Controllers fix problems automatically:
  - Node Controller
  - Deployment Controller
  - ReplicaSet Controller

✅ This enables **self-healing**

STEP 3 Worker Nodes (The Muscle)

Worker nodes are where **applications actually run**.

Each worker node contains:

Linux OS

↓

Container Runtime (Docker / containerd)

↓

kubelet

↓

kube-proxy

↓

Pods → Containers

✅ **kubelet (NODE AGENT)**

- Runs on every worker node
- Talks to API Server

- Pulls container images
- Starts & stops pods
- Reports node health

👉 kubelet is the **bridge** between Kubernetes and containers

---

✅ **kube-proxy (NETWORKING BRAIN)**

- Handles Service networking
- Routes traffic to correct pods
- Uses iptables or IPVS

When traffic hits a Service →
kube-proxy sends it to the right pod.

STEP 4 Pods & Containers (Where Apps Live)

- **Pod = smallest unit in Kubernetes**
- Pod can contain **one or multiple containers**
- Containers in a pod share:
    - IP address
    - Network
    - Storage

Examples:

- Pod 1 → Container1, Container2
- Pod 2 → Container1
- Pod 3 → Container1, Container2

✅ Pods are distributed across worker nodes

STEP 5 Complete Request Flow (MOST IMPORTANT)

This flow is asked in almost **every interview**.

User → kubectl apply

↓

API Server (validates)

↓

ETCD (stores desired state)

↓

Scheduler (chooses node)

↓

kubelet (on worker node)

↓

Container Runtime

↓

Pod & Containers created

✅ Same flow for deployments, scaling, updates, and healing

STEP 6 High Availability (Why 3 Control Plane Nodes?)

- No single point of failure
- Leader election between control plane nodes
- Production stability

Scenarios:

- Control plane node fails → cluster still works
- Worker node fails → pods move to another node

✅ This is **true HA Kubernetes**

STEP 7 How Docker Concepts Fit Here

Earlier Concept	Kubernetes Mapping
Docker Engine	Container runtime on worker node
Containers	Run inside Pods

Earlier Concept	Kubernetes Mapping
Docker Hosts	Worker nodes
Orchestration	Control Plane
High Availability	Multi control plane + nodes

✅ Kubernetes is the **evolution of Docker**

From Architecture → Real AWS Deployment (kops)

Now that you understand **how Kubernetes works**, let's see how we **build it on AWS**.

✅ **Prerequisites for kops (CLEAN & PRACTICAL)**

1️⃣ **AWS Account**

- Required for infrastructure

2️⃣ **Management Server**

- EC2 Ubuntu (t2.micro)
- Used only to:
    - Run kops
    - Run kubectl
    - Manage cluster

3️⃣ **S3 Bucket (State Store)**

k8sb04kopsbucket

Used to store:

- Cluster configuration
- Certificates
- State

---

4 Domain Name

Example:

k8sb26.xyz

Required for:

- Cluster discovery
- API server DNS

---

5 Route53 Hosted Zone

- Create hosted zone for k8sb26.xyz
- kops automatically creates DNS records

---

6 IAM User or Role

Permissions required:

- EC2
- VPC
- IAM
- S3
- Route53

---

7 SSH Keys (Node Access)

ssh-keygen

Used by kops to:

- Access master & worker nodes

---

8 AWS Credentials on Mgmt Server

mkdir ~/.aws

nano ~/.aws/credentials

[default]

aws_access_key_id=XXXXX

aws_secret_access_key=XXXXX

---

9 Install kops

curl -Lo kops https://github.com/kubernetes/kops/releases/latest/download/kops-linux-amd64

chmod +x kops

mv kops /usr/local/bin/

---

10 Install kubectl

curl -LO https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl

chmod +x kubectl

mv kubectl /usr/local/bin/

---

11 Environment Variables

export CLUSTER_NAME=k8sb26.xyz

export AWS_REGION=us-east-1

export KOPS_STATE_STORE=s3://k8sb04kopsbucket

export KUBE_EDITOR=nano

---

12 Deploy Kubernetes Cluster

kops create cluster --name=$CLUSTER_NAME --zones=us-east-1a

kops update cluster --name=$CLUSTER_NAME --yes

kops validate cluster

✅ Kubernetes cluster is LIVE